

Universidad de La Habana
Facultad de Matemática y Computación

Proyecto de Simulación

Cobardía vs altruismo

Estudiante: David Sánchez Iglesias
Grupo: C411
Asignatura: Simulación
Año académico: 4.º año, Curso 2025–2026

Repositorio GitHub:

https://github.com/Davidbestion/Simulation_cowardice_vs_altruism

La Habana, mayo de 2026

Índice

1. Introducción	2
2. Modelo	2
2.1. Entidades y estado del sistema	2
2.2. Dinámica por generación	2
2.3. Estrategias génicas consideradas	3
2.4. Supuestos y alcance	3
3. Implementación	3
3.1. Estructura general del código	3
3.2. Mundo y árboles	4
3.3. Criaturas y genoma	4
3.4. Genes y comportamientos	4
3.5. Ciclo de simulación (por generación)	5
3.6. Asignación de criaturas a árboles	5
3.7. Depredadores y detección	6
3.8. Generación de genomas y experimento	6
3.9. Reproducción y manejo del genoma	6
3.10. Inicialización, semilla y reproducibilidad	6
3.11. Reportes	6
3.12. Notas de extensibilidad y consideraciones de implementación	6
4. Diseño Experimental	7
4.1. Estudio 1: línea base de cobardía pura	7
4.2. Estudio 2: viabilidad del altruismo puro frente a la cobardía	7
4.3. Estudio 3: altruismo selectivo con señal de reconocimiento	7
4.4. Estudio 4: competencia con señal y comportamiento desacoplados	8
5. Resultados	8
5.1. Prueba 1: dinámica de la cobardía pura	8
5.2. Prueba 2: cobardía frente a altruismo puro ($p_{\text{escape}} = 0,50$)	9
5.3. Prueba 3: altruismo puro con alta probabilidad de escape ($p_{\text{escape}} = 0,90$)	9
5.4. Prueba 4: verificación estadística con 30 repeticiones ($p_{\text{escape}} = 0,90$)	10
5.5. Prueba 5: altruismo selectivo con señal de barba verde	10
5.6. Prueba 6: separación de la señal y del comportamiento altruista	12
6. Discusión	13
6.1. Inestabilidad evolutiva del altruismo puro	13
6.2. Efectividad del altruismo selectivo con señal de barba verde	13
6.3. Influencia de la proporción inicial y del coste individual sobre el altruismo selectivo	13
6.4. Robustez del tamaño poblacional	14
7. Conclusiones	14

1. Introducción

Este trabajo estudia, mediante simulación computacional basada en agentes, la competencia evolutiva entre estrategias de cooperación y de conducta egoísta en un entorno con depredación y recursos limitados. El problema central es clásico en biología evolutiva: bajo qué condiciones el altruismo puede mantenerse cuando comparte población con individuos que se benefician sin asumir costes.

El proyecto modela poblaciones de criaturas cuyo comportamiento emerge de su genoma, compuesto por genes que determinan tanto rasgos observables (por ejemplo, la señal de barba verde) como reglas de acción ante depredadores. Sobre esa base se comparan estrategias de cobardía, altruismo puro y altruismo selectivo, analizando su efecto sobre la supervivencia, la reproducción y la frecuencia génica a lo largo del tiempo.

El objetivo del informe es doble: (i) describir con precisión el modelo implementado y el diseño experimental, y (ii) discutir qué patrones son robustos y cuáles dependen de la estocasticidad de ejecuciones individuales. De este modo, el estudio conecta la implementación del simulador con la interpretación evolutiva de los resultados.

2. Modelo

2.1. Entidades y estado del sistema

El sistema está formado por una población de criaturas y un entorno discreto de árboles. Cada criatura posee un *genoma* (lista de genes) del que se derivan sus rasgos físicos y su comportamiento frente al depredador. En la implementación actual, el rasgo físico relevante es `green_beard`, utilizado por estrategias de cooperación selectiva para decidir a quién beneficiar.

El entorno contiene `num_trees` árboles, cada uno con capacidad máxima de 2 criaturas por generación. La presencia de un depredador en cada árbol se modela como una variable Bernoulli independiente con probabilidad `predator_probability`. Por tanto, en cada generación la mortalidad resulta de dos mecanismos: *hambre* (exceso de población respecto a la capacidad total) y *depredación*.

2.2. Dinámica por generación

La simulación opera en generaciones no solapadas. En cada generación se ejecutan, en orden, las siguientes fases:

1. **Asignación a árboles:** las criaturas se distribuyen aleatoriamente en slots de alimentación (capacidad total igual a $2 \times \text{num_trees}$). Si la población excede la capacidad, el excedente muere por hambre.
2. **Encuentro con depredador:** en cada árbol ocupado se determina si hay depredador. Si no lo hay, todos sobreviven. Si lo hay y solo hay un ocupante, ese individuo muere. Si hay dos ocupantes, uno actúa como detector y la resolución depende del comportamiento definido por sus genes.
3. **Reproducción:** los supervivientes generan entre `offspring_min` y `offspring_max` descendientes; los progenitores no pasan a la siguiente generación.
4. **Actualización:** la nueva población queda formada por los descendientes y se registran las métricas de la generación (tamaño poblacional, muertes por hambre, muertes por depredación y frecuencias génicas).

2.3. Estrategias génicas consideradas

Las criaturas diseñadas para este proyecto pueden portar genes que definen su comportamiento ante la presencia de un depredador en el árbol que ocupan. Dichos genes pueden implementar distintas estrategias, que se resumen a continuación:

- **CowardGene**: el detector huye y los demás ocupantes del árbol no sobreviven.
- **AltruistGene**: el detector avisa a todos los demás; su propia supervivencia ocurre con probabilidad `altruist_escape_prob`.
- **SelectiveAltruistGene**: el detector solo beneficia a individuos con `green_beard`.
- **GreenBeardGene**: aporta la señal física `green_beard` sin comportamiento altruista asociado.
- **GreenBeardAltruistGene**: combina señal `green_beard` y altruismo selectivo.

Cuando una criatura posee múltiples genes con comportamiento frente a depredador, se aplica una regla de prioridad (campo `PRIORITY`) y ejecuta el gen de mayor prioridad que produzca una resolución válida.

2.4. Supuestos y alcance

El modelo asume herencia directa del genoma, sin mutación, con apareamiento implícito y generaciones discretas. Asimismo, la asignación a árboles y la presencia de depredador se tratan como procesos aleatorios independientes entre generaciones. Bajo estos supuestos, el simulador no pretende reproducir una especie concreta, sino ofrecer un entorno controlado para estudiar estabilidad o colapso de estrategias cooperativas en función del coste individual, la composición inicial y la estructura de señales de reconocimiento.

3. Implementación

3.1. Estructura general del código

El código principal de la simulación se encuentra en `src/simulation/core` y está organizado por responsabilidad:

- `simulation.py`: definiciones de configuración (`SimulationConfig`), la interfaz `Experiment` y el registro por generación (`GenerationReport`).
- `world.py`: definición de `Tree` y `World` (capacidad de alimentación por árbol).
- `creature.py`: clase `Creature` que encapsula el genoma, rasgos físicos y de comportamiento y la interfaz para responder ante un depredador.
- `genes.py`: implementaciones de los genes (p. ej. `CowardGene`, `AltruistGene`, `SelectiveAltruistGene`, `GreenBeardGene`, `GreenBeardAltruistGene`) y la API que deben seguir.
- `simulator.py`: la lógica del ciclo diurno / por generación y la recogida de `GenerationReport`.

En adición, la carpeta `src/simulation/instances` contiene los experimentos y sus generadores de genomas, y `src/simulation/analysis` las funciones de visualización.

3.2. Mundo y árboles

El entorno se modela con la clase `Tree` (archivo `world.py`). Cada `Tree` tiene:

- `tree_id`: identificador int.
- `max_capacity`: número máximo de criaturas que pueden alimentarse (por defecto 2).
- `has_predator`: bandera booleana que indica si en ese día hubo un depredador.

La clase `World` agrupa la colección de `Tree` y proporciona utilidades como la capacidad total (`total_tree_capacity`). El `Simulator` crea una lista de `Tree` cada generación basada en `cfg.num_trees`.

3.3. Criaturas y genoma

Cada criatura es una instancia de `Creature` (archivo `creature.py`) construida con un `genome`, que es una lista de instancias de `Gene`.

Al inicializarse, `Creature` calcula dos diccionarios derivados iterando sobre su genoma:

- `physical_traits`: resultado de llamar `gene.apply_physical_traits(creature)` para cada gen.
- `behavioral_traits`: resultado de `gene.apply_behavioral_traits(creature)`.

Estos diccionarios permiten consultas rápidas con `has_trait(trait)` y facilitan que genes distintos aporten rasgos (por ejemplo `green_beard`).

Adicionalmente `Creature` expone `has_gene(gene_type)` para comprobar si en el genoma hay una instancia de cierta clase de gen.

3.4. Genes y comportamientos

El contrato de un gen está definido por la clase abstracta `Gene` (archivo `genes.py`). Los puntos clave son:

- `name`: nombre legible del gen (usado en contadores de frecuencia).
- `PRIORITY`: entero que indica prioridad cuando varios genes implementan `predator_behavior`.
- `apply_physical_traits(creature)`: devuelve cambios de rasgos físicos.
- `apply_behavioral_traits(creature)`: devuelve rasgos de comportamiento.
- `predator_behavior(notifier, assigned, cfg)`: opcional; debe devolver (`survivors`, `eaten_count`) o `None` si no aplica.

Implementaciones concretas relevantes:

- `CowardGene` (`PRIORITY=10`): el notificante huye y los demás mueren (salvando al notificante).
- `AltruistGene` (`PRIORITY=20`): al avisar, todos los demás escapan; el notificante escapa con probabilidad `cfg.altruist_escape_prob`.
- `SelectiveAltruistGene` (`PRIORITY=25`): altruismo selectivo solo hacia criaturas con `green_beard`.
- `GreenBeardGene` (`PRIORITY=5`): aporta el rasgo físico `green_beard`.

- **GreenBeardAltruistGene** (PRIORITY=30): combina **green_beard** + comportamiento altruista (prioridad alta).

El diseño de **predator_behavior** permite que si una criatura posee múltiples genes que definen comportamiento ante depredador, **Creature.handle_predator** ordene dichos genes por PRIORITY (descendente) y ejecute en ese orden hasta que uno devuelva una resolución.

3.5. Ciclo de simulación (por generación)

La clase **Simulator** (archivo **simulator.py**) implementa el ciclo principal en **run()**:

1. (Opcional) inicialización del RNG con **cfg.seed** para reproducibilidad.
2. Llamada a **_initialize_population()** que usa **experiment.population_specs** para construir la población inicial (vease más abajo).
3. Bucle de generaciones **for gen in 1..cfg.num_generations**:
 - a) Crear **Tree** objetos y el **World** para esta generación.
 - b) Calcular **total_capacity** sumando **tree.max_capacity**.
 - c) Asignar criaturas a slots de **Tree** (ver sección siguiente). Si la población excede la capacidad, algunos mueren por hambre (**starved**).
 - d) Para cada **Tree** con ocupantes:
 - Determinar si hay depredador: **t.has_predator = random.random() < cfg.predator_probabil**
 - Si no hay depredador, todos los asignados sobreviven.
 - Si hay depredador:
 - Si solo hay un ocupante, ese muere (**eaten += 1**).
 - Si hay más de uno, se elige aleatoriamente un **detector** entre los asignados y se llama **detector.handle_predator(assigned, cfg)**. Esta función delega en los genes (priorizados) y devuelve la lista de **survivors** y **eaten_count**.
 - e) Los supervivientes se reproducen: por cada superviviente se llama **Creature.reproduce()** que genera N descendientes (**random.randint(cfg.offspring_min, cfg.offspring_max)**). Los progenitores no se mantienen (morirían al final del día), por tanto la nueva población está formada solo por los descendientes.
 - f) Se computan las frecuencias genicas en la nueva población y se crea un **GenerationReport** con las métricas de la generación.
 - g) Si la población queda vacía, el bucle termina anticipadamente.

3.6. Asignación de criaturas a árboles

La asignación se realiza en dos modos según la relación entre la población y la capacidad total:

- Si **population_start > total_capacity**: se consideran **starved = population_start - total_capacity**. Se baraja la población (**random.shuffle**) y se seleccionan los **total_capacity** que están permitidos para alimentarse; estos se distribuyen llenando cada árbol hasta su capacidad.
- Si **population_start <= total_capacity**: se construye una lista de **slots** repitiendo cada **tree_id** **max_capacity** veces y se elige aleatoriamente **population_start** slots con **random.sample(slots, k=population_start)**. Cada criatura se asigna a un slot elegido.

Esta estrategia asegura que la asignación sea aleatoria, pero que respete la capacidad por árbol y el caso en que la población exceda dicha capacidad (muerte por hambre).

3.7. Depredadores y detección

La presencia de un depredador en un árbol se decide por evento Bernoulli con probabilidad `cfg.predator_probability` por árbol (se evalúa cada generación). Si hay depredador y hay más de una criatura en el árbol, se escoge un `detector` aleatorio y se delega en `detector.handle_predator` la resolución del enfrentamiento.

`Creature.handle_predator` implementa la lógica de resolución por genes: agrupa los genes que implementan `predator_behavior`, los ordena por `PRIORITY` en orden descendente y ejecuta cada uno hasta obtener una respuesta no nula. Si ningún gen define comportamiento, la implementación por defecto devuelve `([], len(assigned))` (todos muertos).

El formato devuelto por un gen es `(survivors, eaten_count)` donde `survivors` es una lista de referencias a criaturas que escapan y `eaten_count` indica cuántas son comidas por el depredador.

3.8. Generación de genomas y experimento

Cada `Experiment` define `population_specs`, una lista de tuplas `(genome_factory, fraction)` donde `genome_factory()` debe devolver una lista de instancias de `Gene` (el genoma) y `fraction` indica la fracción de la población inicial que tendrá ese genoma. `_initialize_population()` convierte esas fracciones en conteos enteros, corrige errores de redondeo y construye `Creature(genome=...)` para cada individuo.

Ejemplo: un experimento puede usar `lambda: [CowardGene()]` para crear individuos cuya lista `genome` contiene una sola instancia de `CowardGene`.

3.9. Reproducción y manejo del genoma

La función `Creature.reproduce()` crea nuevos objetos `Creature` con el mismo campo `genome` que el progenitor. En la implementación actual, la lista del genoma se pasa tal cual a los descendientes; esto funciona si los genes son inmutables, pero se debería copiar el genoma si en el futuro se permitieran mutaciones a nivel de instancia.

3.10. Inicialización, semilla y reproducibilidad

La clase `SimulationConfig` define los parámetros controlables: `num_trees`, `num_generations`, `initial_population`, `predator_probability`, `offspring_min`, `offspring_max` y `seed` (entre otros). `Simulator.run()` llama a `random.seed(cfg.seed)` cuando `seed` no es `None`, lo que permite reproducir ejecuciones.

En la utilidad `run.py` se implementa la lógica para repetir un experimento varias veces (`-repeats`) y para controlar si cada repetición debe ser determinista (semillas `base + i`) o no (semillas `None` para variar con el RNG del sistema).

3.11. Reportes

Por cada generación `Simulator` construye un `GenerationReport` (definido en `simulation.py`) que contiene: `generation`, `population_start`, `starved`, `eaten`, `survived`, `population_end` y `gene_frequencies` (mapa `nombre_gene -> fracción relativa`). Estos objetos se usan luego para generar las figuras y el análisis agregado.

3.12. Notas de extensibilidad y consideraciones de implementación

- La separación de responsabilidades facilita añadir nuevos genes implementando `predator_behavior` o `apply_*_traits`.

- Si se desea permitir la mutación de genomas es recomendable clonar la lista `genome` en `Creature.reproduce()` para evitar aliasing entre instancias.
- La asignación actual de depredadores es independiente por árbol y por día; puede extenderse para modelar depredadores locales con distribuciones no Bernoulli.

4. Diseño Experimental

El diseño experimental sigue una lógica de complejidad creciente: se parte de un escenario homogéneo de referencia, se introduce progresivamente la competencia entre estrategias, y se añaden mecanismos de reconocimiento y cooperación selectiva para examinar bajo qué condiciones el altruismo puede mantenerse evolutivamente. Los parámetros comunes a todas las simulaciones se recogen en la Tabla 1.

Cuadro 1: Parámetros fijos del entorno de simulación.

Parámetro	Valor
Número de árboles	25
Población inicial	80 individuos
Probabilidad de depredador por árbol	0,30
Crías por superviviente por generación	1–2
Semilla aleatoria (<i>seed</i>)	42

4.1. Estudio 1: línea base de cobardía pura

Objetivo. Caracterizar la dinámica poblacional y de supervivencia bajo una estrategia única para disponer de una referencia con la que comparar los estudios posteriores.

Condiciones. Se simula una población homogénea de individuos portadores de **CowardGene**, con 100 generaciones y $p_{\text{escape}} = 0,50$ (configuración por defecto). Se realiza una única ejecución, suficiente para observar la dinámica estacionaria.

4.2. Estudio 2: viabilidad del altruismo puro frente a la cobardía

Objetivo. Determinar si el altruismo puro puede coexistir con o desplazar a la cobardía, y cómo varía ese resultado al modificar la ventaja individual del altruista (p_{escape}).

Condiciones. La población inicial se divide al 50 % entre **CowardGene** y **AltruistGene**. El individuo altruista avisa a todos los compañeros de árbol al detectar un depredador, incurriendo en el riesgo de ser comido con probabilidad $1 - p_{\text{escape}}$. Se contemplan dos valores del parámetro p_{escape} :

- $p_{\text{escape}} = 0,50$, durante 100 generaciones (1 ejecución exploratoria).
- $p_{\text{escape}} = 0,90$, durante 200 generaciones. Para este segundo valor, se realizan primero una ejecución individual para observar la trayectoria concreta y a continuación 30 ejecuciones independientes para estimar la variabilidad entre runs y confirmar la robustez estadística del resultado.

4.3. Estudio 3: altruismo selectivo con señal de reconocimiento

Objetivo. Estudiar si el altruismo puro puede volverse evolutivamente estable cuando se lo dota de un mecanismo de reconocimiento: la señal física de barba verde (*green beard*), que permite al individuo altruista cooperar exclusivamente con otros portadores de esa misma señal. Una vez

confirmada la viabilidad del mecanismo en la distribución por defecto (50 % de altruistas), se examina cómo influyen dos factores en su estabilidad: la proporción inicial de altruistas en la población y el coste individual del altruismo (p_{escape}).

Condiciones. En todas las condiciones de este estudio el `GreenBeardAltruistGene` mantiene la señal y el comportamiento altruista completamente acoplados: todo individuo con barba verde es altruista selectivo y viceversa. Se varían dos factores de forma cruzada, dando lugar a seis condiciones simuladas con 30 ejecuciones cada una (200 generaciones):

1. **Probabilidad de escape** (p_{escape}): 0,10 (coste alto del altruismo), 0,50 (coste moderado), 0,90 (coste bajo).
2. **Proporción inicial de altruistas selectivos:** 50 % de altruistas frente a 10 % de altruistas (con el 90 % o el 50 % restante compuesto por cobardes puros, respectivamente).

Esta combinación 3×2 permite separar el efecto de p_{escape} del efecto de la proporción inicial sobre la dinámica de fijación o extinción del gen altruista.

4.4. Estudio 4: competencia con señal y comportamiento desacoplados

Objetivo. Examinar qué sucede cuando la señal de barba verde y la conducta altruista existen como genes independientes que pueden segregarse por separado en la población. En este escenario ninguna estrategia altruista tiene garantía de que avisar a un portador de la señal implique cooperación recíproca.

Condiciones. La población inicial se distribuye equitativamente al 25 % entre cuatro fenotipos: `CowardGene` (cobarde puro, sin señal), `GreenBeardGene+CowardGene` (porta la señal pero actúa como cobarde), `SelectiveAltruistGene` (altruismo selectivo hacia portadores de barba verde, sin poseer la señal propia), y `GreenBeardAltruistGene` (porta la señal y el altruismo selectivo acoplados). Se realizan 30 ejecuciones de 400 generaciones con $p_{\text{escape}} = 0,50$, extendiendo el horizonte temporal para observar posibles dinámicas de largo plazo.

5. Resultados

5.1. Prueba 1: dinámica de la cobardía pura

La población homogénea de cobardes (`Experiment1`, `default_config`, 1 ejecución) muestra una dinámica estable a lo largo de las 100 generaciones simuladas (Figura 1). El tamaño poblacional al final de cada generación oscila entre aproximadamente 52 y 80 individuos, reflejando el equilibrio dinámico entre reproducción y mortalidad por depredación y por hambre. Este resultado establece la línea base: bajo cobardía pura la especie ni se extingue ni crece ilimitadamente, y la frecuencia del gen cobarde se mantiene en el 100 % durante toda la simulación.

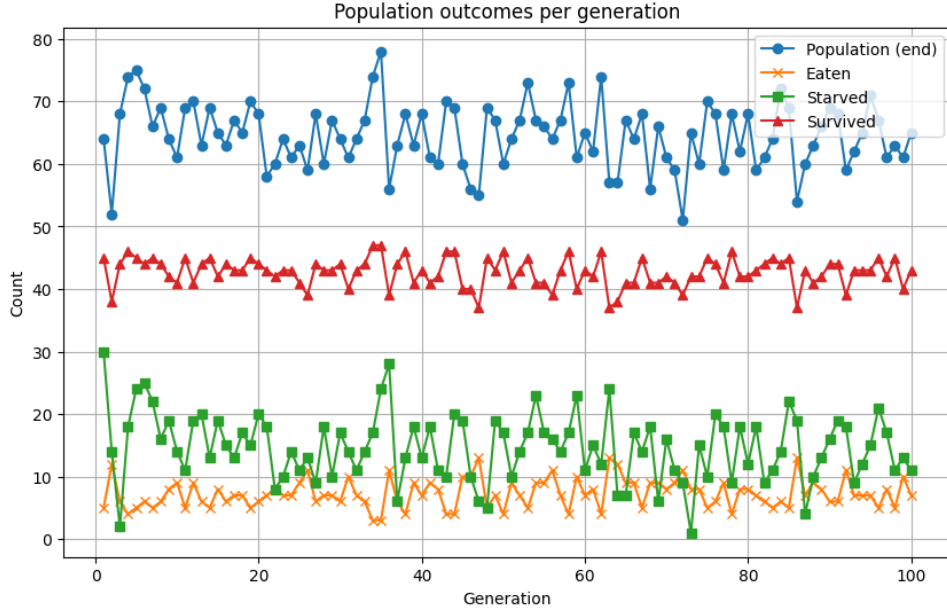
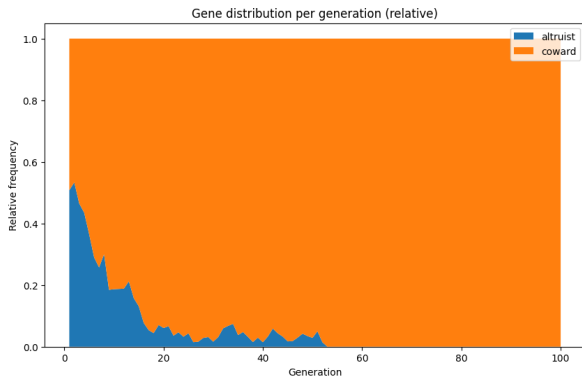


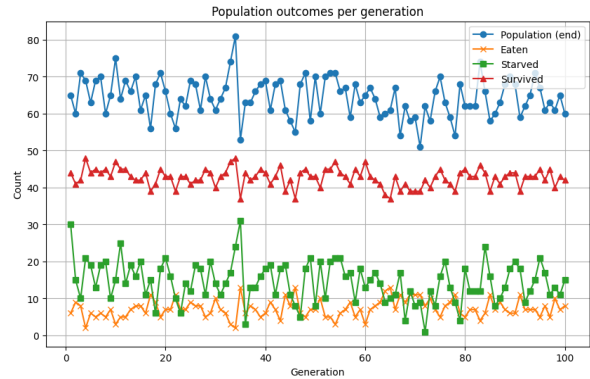
Figura 1: Prueba 1 (`Experiment1`, `default_config`): dinámica poblacional durante 100 generaciones con cobardía pura.

5.2. Prueba 2: cobardía frente a altruismo puro ($p_{\text{escape}} = 0,50$)

Introduciendo un 50 % de altruistas puros (`AltruistGene`) con la configuración por defecto (`default_config`), el gen altruista desaparece de la población con rapidez (Figura 2). La frecuencia del gen altruista cae desde el ≈ 50 % inicial hasta prácticamente cero en torno a la generación 50, tras lo cual la población queda compuesta exclusivamente por cobardes y su dinámica reproduce la de la Prueba 1. El altruista puro avisa a todos los individuos del árbol, incluyendo cobardes, que se benefician del aviso sin asumir ningún coste, lo que confiere a los cobardes una aptitud relativa superior.



(a) Distribución genética por generación.



(b) Dinámica poblacional.

Figura 2: Prueba 2 (`Experiment2`, `default_config`, 1 ejecución, $p_{\text{escape}} = 0,50$).

5.3. Prueba 3: altruismo puro con alta probabilidad de escape ($p_{\text{escape}} = 0,90$)

Aumentar p_{escape} al 90 % (`high_escape_prob`) produce un resultado llamativo en esta ejecución individual (Figura 3): lejos de seguir el patrón de la Prueba 2, la frecuencia del altruista oscila erráticamente durante las primeras ~ 70 generaciones, alternando entre dominio de cobardes y repuntes del altruista, lo que sugiere inestabilidad dinámica. A partir de la generación 70 el

altruista comienza a imponerse de forma sostenida y alrededor de la generación 110 los cobardes se extinguen; el altruista termina fijándose en el 100 % de la población. Este resultado contrasta radicalmente con la Prueba 2 y plantea la pregunta de si, con p_{escape} suficientemente alto, el altruismo puro puede imponerse de forma general, o si la victoria del altruista en esta ejecución es un mero efecto del azar.

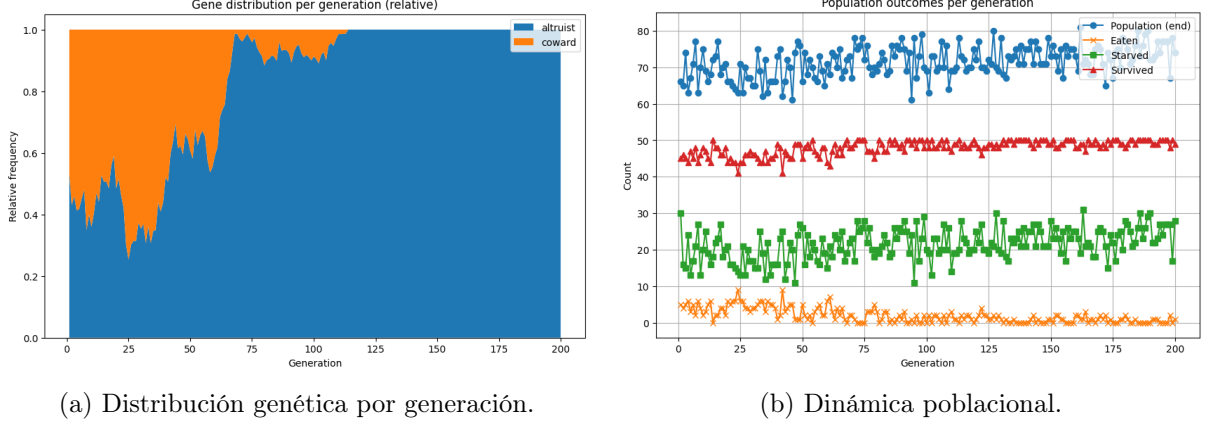


Figura 3: Prueba 3 (Experiment2, high_escape_prob, 1 ejecución, $p_{\text{escape}} = 0,90$).

5.4. Prueba 4: verificación estadística con 30 repeticiones ($p_{\text{escape}} = 0,90$)

Para determinar si la victoria del altruista observada en la Prueba 3 constituye el comportamiento general o es un resultado puntual debido al azar, se replicó la misma condición exacta 30 veces. La Figura 4 muestra la evolución genética agregada (media de los 30 runs). El gen altruista decrece de forma sostenida desde el 50 % inicial hasta situarse en torno al 15–20 % al cabo de apenas 100 generaciones, sin que la media de los 30 runs muestre ningún repunte ni tendencia hacia la fijación. La victoria del altruista observada en la Prueba 3 es por tanto un resultado excepcional producto de la estocasticidad del modelo, no el comportamiento general: promediando las trayectorias, el altruismo puro está destinado a desaparecer incluso con $p_{\text{escape}} = 0,90$. El tamaño de la población (Figura 4b) permanece estable y equivalente al de la Prueba 1.

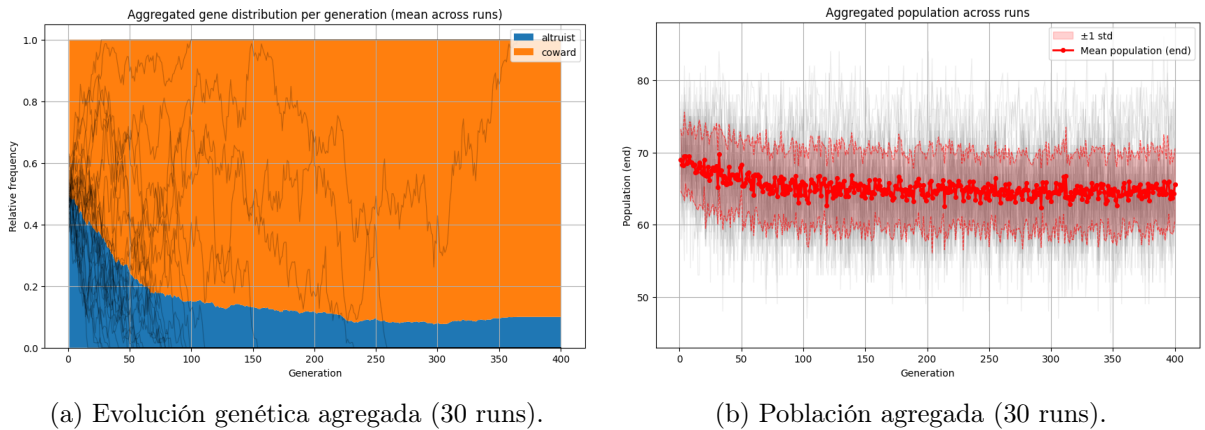


Figura 4: Prueba 4 (Experiment2, high_escape_prob, 30 repeticiones, $p_{\text{escape}} = 0,90$).

5.5. Prueba 5: altruismo selectivo con señal de barba verde

Esta prueba estudia la viabilidad del `GreenBeardAltruistGene` bajo seis condiciones, resultado de cruzar tres valores de p_{escape} (0,10 / 0,50 / 0,90) con dos proporciones iniciales de

altruistas: 50 % (mayoritaria) y 10 % (minoritaria), siendo el resto de la población cobardes puros en ambos casos. En todo momento la señal de barba verde y el comportamiento altruista van acoplados: no existen individuos cobardes con barba verde.

Proporción inicial del 50 % de altruistas selectivos

Con la mitad de la población inicial compuesta por altruistas selectivos, el `GreenBeardAltruistGene` se fija en la población en pocas generaciones bajo las tres probabilidades de escape (Figura 5). A $p_{\text{escape}} = 0,50$ la fijación resulta evidente, prácticamente extinguiendo a los cobardes en el transcurso de 100 días; a $p_{\text{escape}} = 0,10$ la ventaja de los altruistas es mucho menor, y los datos reflejan esto mostrando un crecimiento mucho más lento del altruista, aunque sostenido; y a $p_{\text{escape}} = 0,90$ el altruista se impone con rapidez, extinguiendo a los cobardes en 100 generaciones. El altruismo selectivo elimina el coste de cooperar con cobardes al avisar únicamente a otros portadores de la señal, generando una ventaja que supera a la cobardía con independencia del coste individual del altruismo.

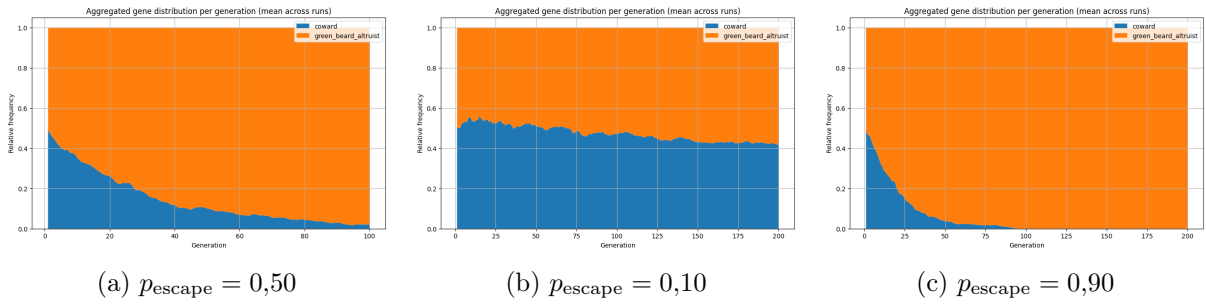


Figura 5: Prueba 5, proporción inicial del 50 % de altruistas selectivos (30 runs c/u). El gen `GreenBeardAltruistGene` se fija en la población en todos los casos.

Proporción inicial del 10 % de altruistas selectivos

Reducir la presencia inicial del altruista al 10 % altera sustancialmente los resultados (Figura 6). A $p_{\text{escape}} = 0,10$ el gen cobarde domina completamente y el altruista selectivo aunque no desaparece en todas las ejecuciones, sí se puede observar cómo la mayoría de las trayectorias se extinguen relativamente rápido, y las que persisten muestran una evolución errática con oscilaciones pronunciadas, sin tendencia clara hacia la fijación. A $p_{\text{escape}} = 0,50$ se observa cómo en la mayoría de las ejecuciones se extingue el altruista si su población no crece lo suficiente alrededor de los primeros 50 días, pero en las ejecuciones donde el altruista logra superar esa fase inicial de vulnerabilidad, se impone con rapidez superando a los cobardes y fijándose en la población. A $p_{\text{escape}} = 0,90$ se puede observar cómo en alrededor de la mitad de las ejecuciones se extingue el altruista en la fase inicial, pero en la otra mitad el altruista logra superar esa fase y se impone con rapidez, fijándose en la población. Estos resultados muestran que la viabilidad del altruismo selectivo depende conjuntamente de la probabilidad que tienen de escapar de los depredadores (p_{escape}) y de la proporción inicial: a coste individual bajo, el altruista puede prosperar incluso desde una posición minoritaria; pero a coste alto, la masa crítica inicial resulta determinante.

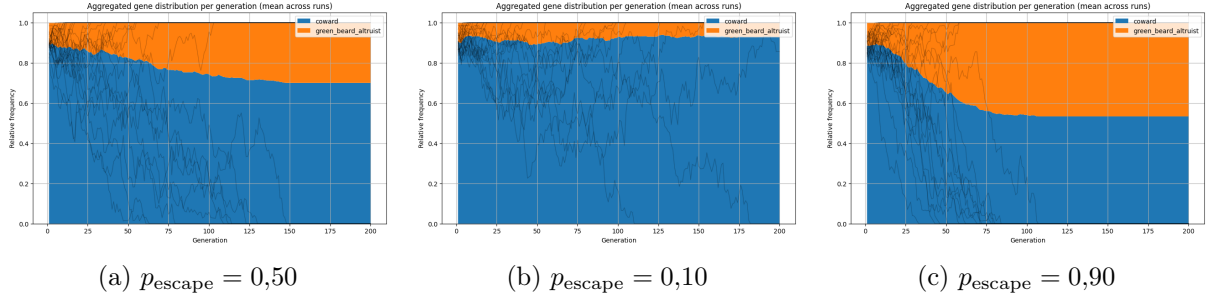


Figura 6: Prueba 5, proporción inicial del 10 % de altruistas selectivos (30 runs c/u). La viabilidad del gen altruista depende del coste individual (p_{escape}).

Curiosidad: dinámica de población con 10 % inicial de altruistas

Como complemento descriptivo, la Figura 7 muestra la evolución del tamaño poblacional agregado (media y dispersión entre runs) para el mismo bloque de comparaciones de la Figura 6, es decir, manteniendo 10 % de altruistas selectivos iniciales y variando p_{escape} . Se aprecia que con $p_{\text{escape}} = 0,90$ la población media total presenta una tendencia ascendente más marcada a lo largo de las generaciones, mientras que en $p_{\text{escape}} = 0,10$ y $p_{\text{escape}} = 0,50$ la evolución es más plana y con menor incremento neto. Esto se debe a que, al haber más ejecuciones donde el altruista logra imponerse a los cobardes, el beneficio mutuo entre altruistas se traduce en una mayor supervivencia y reproducción, lo que impulsa un crecimiento poblacional más pronunciado. En contraste, con $p_{\text{escape}} = 0,10$ la mayor parte de las ejecuciones terminan dominadas por cobardes, lo que limita el crecimiento poblacional y mantiene una dinámica más estable.

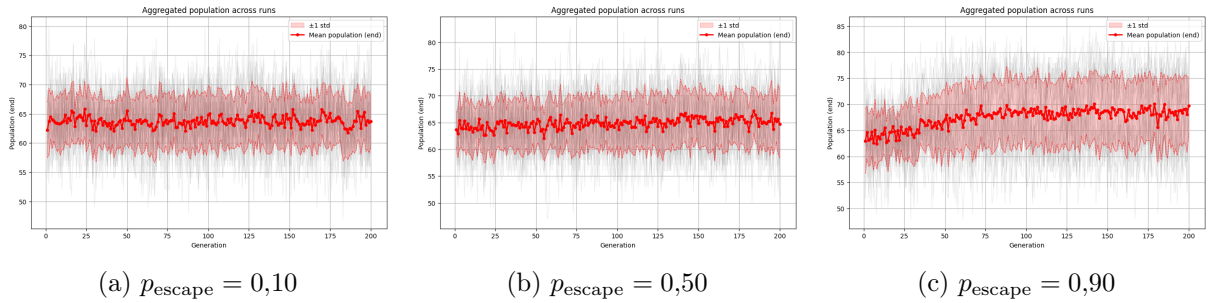
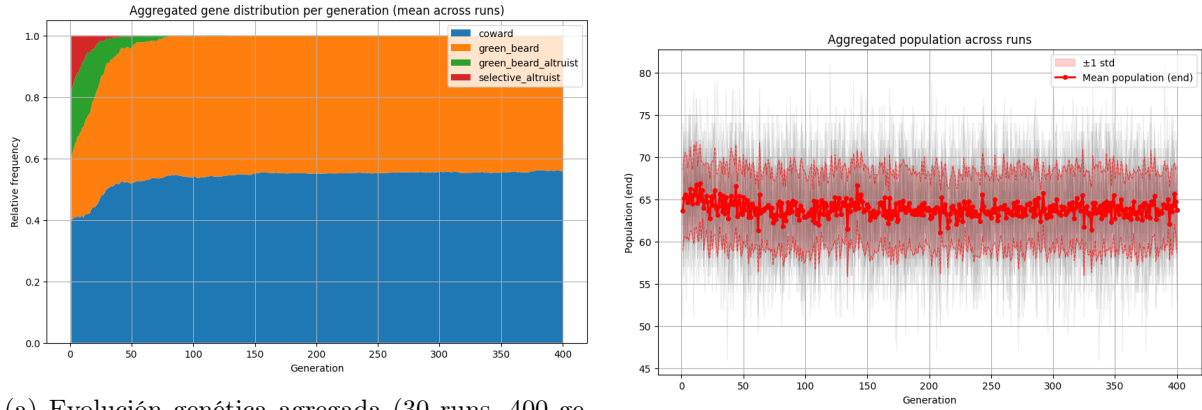


Figura 7: Dinámica poblacional agregada en Prueba 5 con 10 % inicial de altruistas selectivos (30 runs c/u). A mayor p_{escape} se observa un incremento más claro de la población media final.

5.6. Prueba 6: separación de la señal y del comportamiento altruista

La Prueba 6 desacopla explícitamente la señal de barba verde del comportamiento altruista mediante Experiment4, donde GreenBeardGene y SelectiveAltruistGene coexisten como genes separados junto con GreenBeardAltruistGene y CowardGene (25 % cada uno, 400 generaciones). La Figura 8 muestra que tanto SelectiveAltruistGene como GreenBeardAltruistGene desaparecen antes de la generación 50, mientras que la población converge a un equilibrio entre cobarde puro (≈ 55 %) y cobarde con barba verde (≈ 45 %). La barba verde pierde toda utilidad como señal de reconocimiento al no garantizar el comportamiento cooperativo: los altruistas que la usan como criterio de selección acaban cooperando con cobardes barbudos, pagando un coste sin obtener beneficio. El tamaño de la población (Figura 8b) permanece estable en todos los casos, igual que en pruebas anteriores.



(a) Evolución genética agregada (30 runs, 400 generaciones).

(b) Población agregada (30 runs, 400 generaciones).

Figura 8: Prueba 6 (`Experiment4`, `default_config`, 30 repeticiones). Las estrategias altruistas se extinguen antes de la generación 50.

6. Discusión

6.1. Inestabilidad evolutiva del altruismo puro

Las Pruebas 2, 3 y 4 demuestran que el altruismo puro (`AltruistGene`) es evolutivamente inestable cuando compite con la cobardía, con independencia de la probabilidad de escape del altruista. El altruista puro avisa a todos los individuos de su árbol, incluyendo cobardes, que obtienen el beneficio del aviso sin asumir ningún coste. Esto confiere a los cobardes una aptitud relativa superior y desplaza al altruista de la población, en plena concordancia con la teoría de la selección individual [1]. Aumentar p_{escape} al 90 % (Prueba 3 y 4) solo ralentiza el proceso sin revertirlo.

6.2. Efectividad del altruismo selectivo con señal de barba verde

Cuando la señal de barba verde está acoplada al comportamiento altruista y no existen explotadores de la señal, el altruismo selectivo es capaz de invadir y fijar una población mayoritariamente cobarde (Prueba 5, sin barba verde adicional). Esto ocurre porque el `GreenBeardAltruistGene` dirige el coste del altruismo exclusivamente hacia otros portadores de la señal, que a su vez también son altruistas; el beneficio mutuo genera una ventaja de aptitud sobre los cobardes. Este resultado reproduce el mecanismo de la barba verde de Dawkins [2] en un contexto computacional.

6.3. Influencia de la proporción inicial y del coste individual sobre el altruismo selectivo

La Prueba 5 revela que la viabilidad del altruismo selectivo no sólo depende del mecanismo de reconocimiento, sino de la interacción entre la proporción inicial de altruistas y el coste individual del comportamiento (p_{escape}). Cuando los altruistas parten de una posición mayoritaria (50 %), el gen se fija con independencia de p_{escape} : la masa crítica inicial es suficiente para que la cooperación selectiva genere un beneficio neto superior a la cobardía en cualquier escenario de coste. Sin embargo, cuando parten de una minoría (10 %), el resultado depende fuertemente de p_{escape} : a coste alto (0,10) el altruista se extingue porque la escasez de portadores de la señal hace que cada encuentro altruista sea más frecuentemente con un cobarde que con otro altruista, mientras que a coste bajo (0,90) el altruista logra mantenerse en equilibrio estable. Este resultado ilustra la importancia del efecto de masa crítica en la evolución de la cooperación: una

estrategia cooperativa puede ser localmente estable por encima de cierto umbral de frecuencia pero vulnerable a la extinción si parte de una proporción demasiado reducida [1].

La desactivación total del acoplamiento entre señal y comportamiento (Prueba 6) produce el colapso irreversible de ambas estrategias altruistas antes de la generación 50, confirmando que el mecanismo de barba verde sólo es evolutivamente estable cuando la señal implica de forma fiable el comportamiento cooperativo [3].

6.4. Robustez del tamaño poblacional

Una observación transversal a todas las pruebas es que la composición genética determina la *estructura* de la población (qué estrategia domina) pero no su *tamaño*. En todos los escenarios la población oscila en un rango similar al de la Prueba 1, determinado fundamentalmente por la capacidad del entorno (25 árboles) y la tasa reproductiva (1–2 crías por superviviente).

7. Conclusiones

1. El altruismo puro no es evolutivamente estable en el modelo implementado: desaparece de la población independientemente de la probabilidad de escape del altruista (Pruebas 2–4).
2. El altruismo selectivo mediado por la señal de barba verde puede fijarse en la población cuando los altruistas parten de una proporción inicial mayoritaria (50 %), con independencia del coste individual del altruismo (Prueba 5).
3. Cuando los altruistas parten de una proporción inicial reducida (10 %), la viabilidad del mecanismo depende del coste individual: a p_{escape} baja, el gen altruista se extingue; a p_{escape} alta, persiste en un equilibrio estable (Prueba 5). Esto sugiere la existencia de un umbral de masa crítica por debajo del cual el altruismo selectivo no puede prosperar.
4. Cuando la señal y el comportamiento altruista se desacoplan como genes independientes, ambas estrategias altruistas (`GreenBeardAltruistGene` y `SelectiveAltruistGene`) se extinguen antes de la generación 50 y la población converge a cobardes puros o barbudos (Prueba 6).
5. El tamaño de la población es robusto frente a la composición genética y queda determinado por los parámetros ecológicos del entorno.

Los resultados obtenidos sugieren que la evolución del altruismo requiere mecanismos de reconocimiento que sean difíciles de falsificar —como el parentesco genético real o la reciprocidad en interacciones repetidas—, ninguno de los cuales ha sido modelado en el presente trabajo y constituye una línea natural de extensión futura.

Referencias

- [1] W.D. Hamilton, *The genetical evolution of social behaviour I & II*, Journal of Theoretical Biology, 7(1):1–52, 1964.
- [2] R. Dawkins, *The Selfish Gene*, Oxford University Press, Oxford, 1976.
- [3] A. Zahavi, *Mate selection—a selection for a handicap*, Journal of Theoretical Biology, 53(1):205–214, 1975.